Stackademic · Follow publication

Member-only story

🚀 9 RAG Architectures Every AI Engineer Should Actually Understand (Not Just Memorize)

4 min read · May 3, 2026

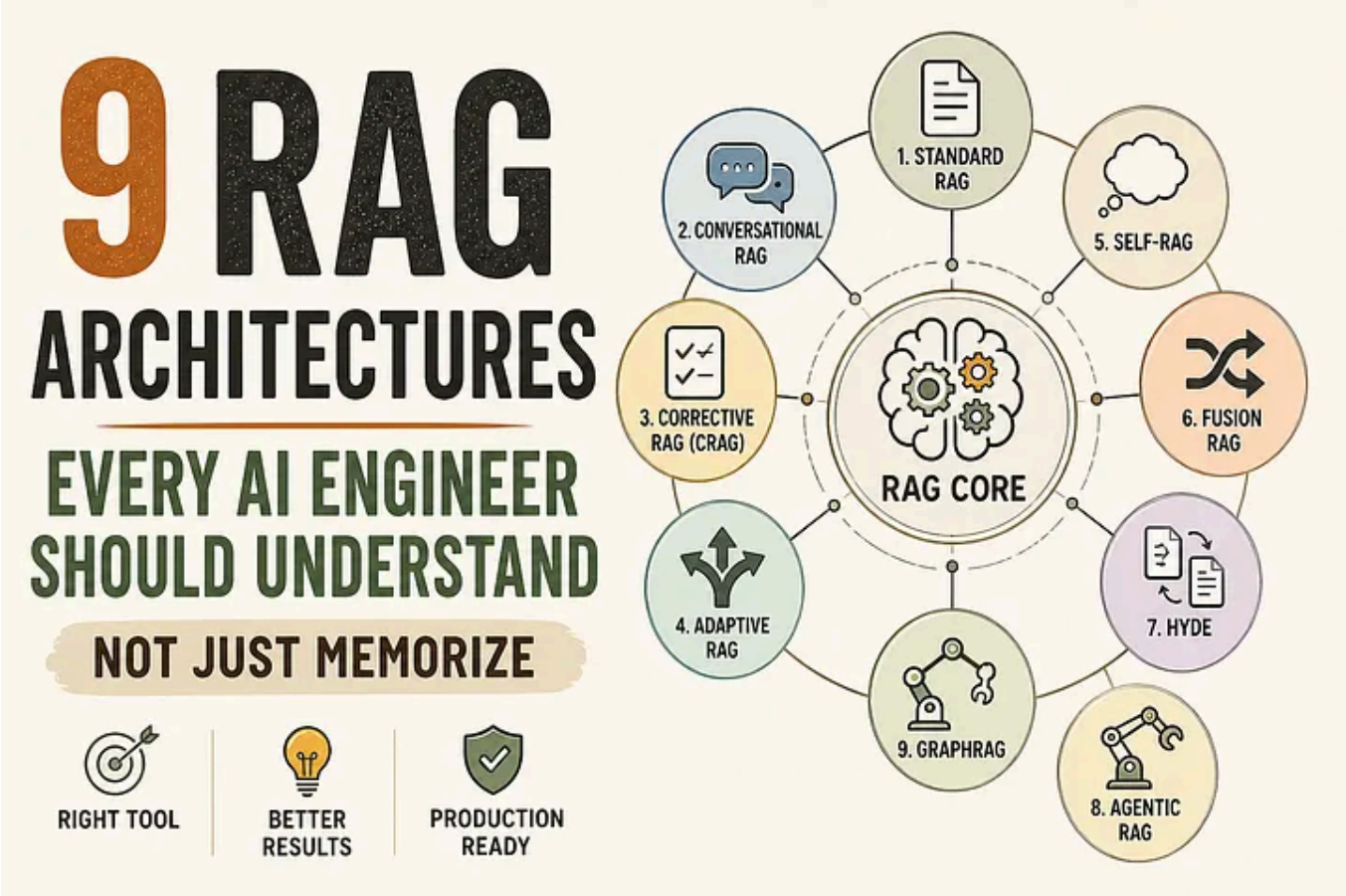
Sachin Kasana Follow

Listen Share More

If you’re building anything with LLMs right now, you’ve probably heard this a hundred times:

> “Just add RAG.”

Sounds simple… until it doesn’t work in production.



Hallucinations. Irrelevant context. Slow responses.
And suddenly your “smart AI” feels… dumb.
Non members can read **here**

The problem isn’t RAG itself.

👉 The problem is using the **wrong RAG architecture for the job.**

So instead of giving you another theoretical breakdown, let’s walk through **what each RAG type actually does, when it breaks, and when you should use it.**

🧠 First — What is RAG (in plain terms)?

At its core, RAG is just:

```
User Query → Retrieve Data → Feed to LLM → Generate Answer
```

That’s it.

But the *magic (and problems)* happen in:

- how you retrieve
- what you retrieve
- how you validate
- and how the model reasons

That's where these 9 architectures come in.

🔥 1. Standard RAG (The Starting Point)

This is what 90% of tutorials show.

- Embed documents
- Store in vector DB
- Retrieve top-k chunks
- Send to LLM

👉 Works well for:

- FAQs
- internal docs
- basic search

👉 Breaks when:

- query is complex
- context is noisy
- reasoning is required

Reality check:
Good for MVPs. Not enough for serious systems.

💬 2. Conversational RAG

Now we add memory.

Instead of treating every query independently, it uses:

- chat history
- previous context

👉 Works well for:

- chatbots
- support systems
- assistants

👉 Breaks when:

- long conversations → context bloat
- irrelevant history pollutes answers

Key insight:
Memory helps… but *too much memory hurts.*

🛠 3. Corrective RAG (CRAG)

This one adds something most systems ignore:

👉 **A validation layer**

Flow becomes:

1. Retrieve documents
2. Evaluate their quality
3. Fix or re-retrieve if needed

👉 Works well for:

- finance
- legal
- compliance systems

👉 Why it matters:

Most RAG failures aren't generation issues.
They're **retrieval failures.**

CRAG fixes that.

⚡ 4. Adaptive RAG

Not all queries are equal.

Some are simple:

> "What is Redis?"

Some are complex:

> "Compare Redis vs Kafka for event-driven systems"

Adaptive RAG decides:

- how many sources to retrieve
- how deep to search
- whether to use multiple strategies

👉 Think of it like:
A smart backend router for queries.

🧠 5. Self-RAG

Now things get interesting.

The model:

- generates an answer
- critiques itself
- improves the response

👉 Think:
"LLM doing code review on itself"

👉 Works well for:

- long-form answers
- research

- content generation

👉 Tradeoff:

- better quality
- higher latency

🔀 6. Fusion RAG

Instead of asking **one query**, it asks multiple.

Example:

User asks:

> "How to scale Node.js app?"

System generates:

- "Node.js scaling techniques"
- "horizontal scaling node"
- "load balancing node apps"

Then:

- retrieves from all
- merges results
- ranks them

👉 Result:
Much better coverage.

👉 This is **criminally underrated in production systems.**

🧪 7. HyDE (Hypothetical Document Embedding)

This one feels weird… but works surprisingly well.

Instead of searching directly, it:

1. Generates a **fake ideal answer**
2. Embeds that
3. Retrieves based on it

👉 Why it works:
The generated answer is often closer to relevant documents than the raw query.

👉 Best for:

- vague queries
- poorly structured input

🤖 8. Agentic RAG

This is where RAG meets agents.

Instead of a single flow, you get:

- planning
- tool usage
- multi-step reasoning

Example:
User asks:

> "Analyze sales drop and suggest strategy"

Agent might:

1. Fetch data
2. Run analysis
3. Retrieve docs
4. Generate insights

👉 This is not just RAG.

This is **AI systems thinking and acting.**

🕸️ 9. GraphRAG

Traditional RAG uses chunks.

GraphRAG uses:
👉 **relationships**

Instead of:

- “random paragraph retrieval”

You get:

- entities
- connections
- structured knowledge

👉 Example:

- “Node.js → event loop → async → performance”

👉 Works best for:

- enterprise knowledge
- research
- connected data

⚡ The Truth Most Articles Won’t Tell You

No real system uses just one.

Production setups usually look like:

🧩 Real-world stack:

- Retrieval → Fusion RAG
- Validation → CRAG
- Routing → Adaptive RAG
- Reasoning → Agentic layer

👉 RAG is not a feature.
👉 It’s a **pipeline architecture.**

🛠️ If You're Building (Practical Roadmap)

Let's keep this grounded.

✅ Level 1 (MVP)

- Standard RAG
- good chunking
- add reranker

⚙️ Level 2 (Production)

- Fusion RAG (multi-query)
- CRAG (validation layer)

🚀 Level 3 (Advanced Systems)

- Agentic RAG
- GraphRAG (if data is relational)

💡 Final Thought

Most developers try to fix hallucinations by:

- switching models
- increasing tokens

But the real fix is almost always:

> **Better retrieval architecture.**

👉 Don't ask: "Should I use RAG?"
👉 Ask: "Which RAG architecture fits my problem?"

That's the difference between:

- a demo
- and a production-grade AI system

AI Rag AI Agent Programming Architecture

Follow

Published in Stackademic

83K followers · Last published 8 hours ago

Stackademic is a learning hub for programmers, devs, coders, and engineers. Our goal is to democratize free coding education for the world.

Follow

Written by Sachin Kasana

276 followers · 90 following

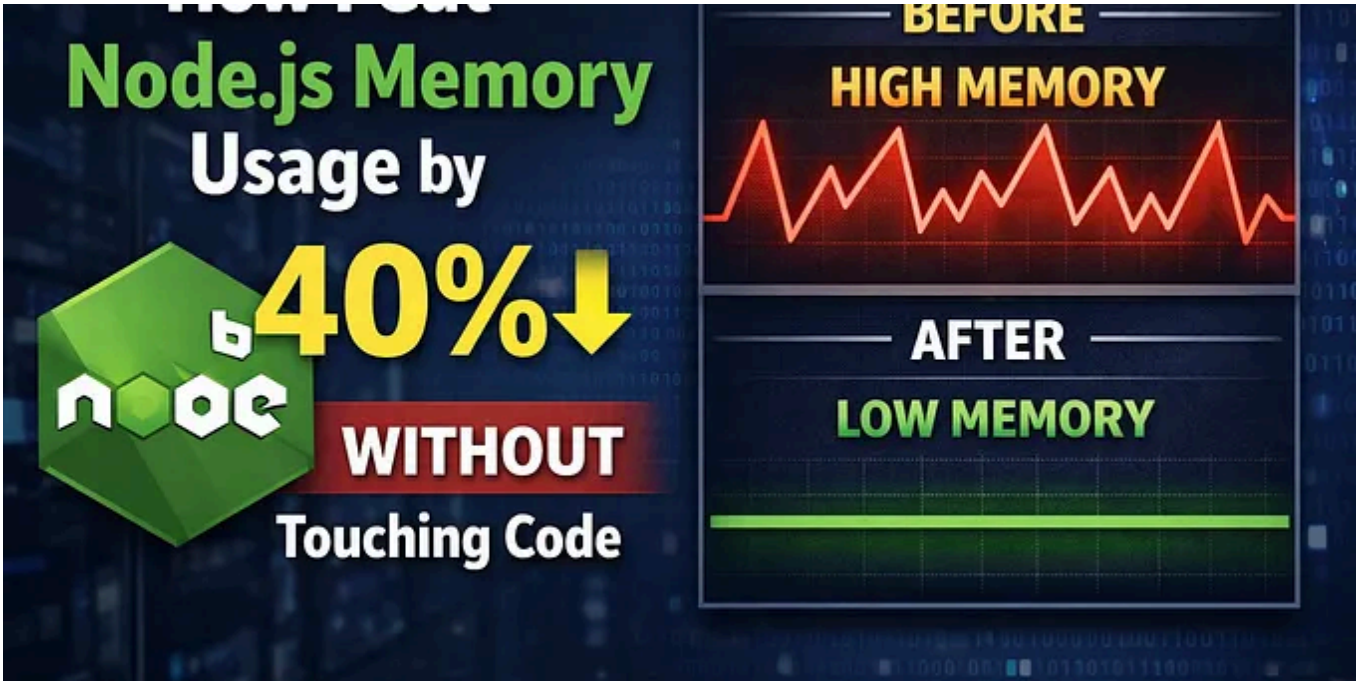
Principal Engineer & Architect | Node.js, React, AWS I build tools, write about clean code, and simplify system design. 🌐
Portfolio: sachinkasana-dev.vercel.app

No responses yet

b Datachamp

What are your thoughts?

More from Sachin Kasana and Stackademic

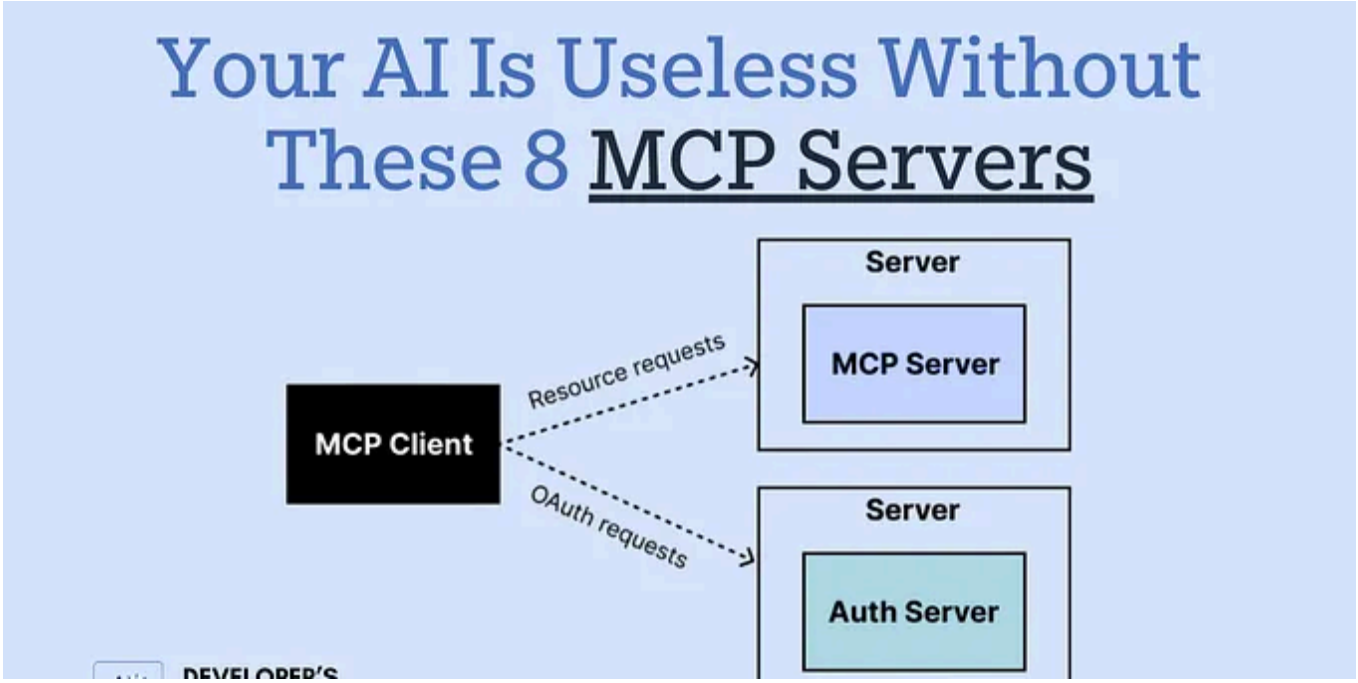


In Stackademic by Sachin Kasana · Mar 20

How I Cut Node.js Memory Usage by 40% Without Touching the Code

We've all seen this.

133



In Stackademic by Usman Writes · Feb 26

Your AI Is Useless Without These 8 MCP Servers — Most Developers Have Never Heard of Them

Two engineers. The same AI model. One copy-pastes files all day. The other connects tools. One is chatting. The other is building.

1.5K 29

In Stackademic by Gwang-Jin · Mar 30

Make Your Home Computer Reachable From Anywhere (No Public IP, No Router Touching) — The “Paranoid”...

The self-hosted, “I don’t want to trust anyone” edition: Headscale + Tailscale + NoMachine

713 16

In Front-end World by Sachin Kasana · Apr 21

🚀 10 Node.js Tools I Use in Production (That Most Devs Ignore)

Most Node.js apps don't struggle because of bad code.

121 2

See all from Sachin Kasana

See all from Stackademic